Check for updates

IET Communications

WILEY

The Institution of Engineering and Technology

ORIGINAL RESEARCH OPEN ACCESS

A Queue-Aware Extension to BBR for Low-Latency Transport in Video Applications

Ziyi Li | Jiao Zhang | Yuqing Wang | Peisheng Guo | Jiafeng Jiang

Telecommunication Engineering, Beijing University of Posts and Telecommunications, Beijing, China

Correspondence: Jiao Zhang (jiaozhang@bupt.edu.cn)

Received: 18 November 2025 | **Revised:** 26 January 2026 | **Accepted:** 12 February 2026

ABSTRACT

Modern model-based congestion control protocols such as BBR achieve high throughput by accurately estimating bottleneck bandwidth and RTT. However, during ProbeBW phases, bandwidth ramp-up can introduce transient queues that accumulate over successive cycles, leading to tail latency increase and degraded performance for delay-sensitive applications like video streaming. In this paper, we propose active queue debt compensation (AQDC-BBR), a lightweight extension to BBR that explicitly quantifies and models the queue debt incurred during bandwidth probing and compensates it in subsequent phases. We evaluated AQDC-BBR in both NS-3 simulations and Mininet video streaming experiments. The results show that AQDC-BBR reduces tail latency by 26% and 25% compared to Copa and PCC, respectively, and achieves a reduction of more than 25% compared to BBR and its variants, while maintaining stable high throughput.

1 | Introduction

Modern interactive applications, such as real-time video conferencing, live streaming and especially emerging edge-based interactive services like cloud gaming, are placing stringent demands on congestion control algorithms (CCAs) [1, 2]. This new generation of services, which is rapidly coming to age, has attracted considerable attention from both academic and industrial communities [3–6]. To provide a seamless interaction between servers and users, the streaming needs to be delivered with low latency.

Consequently, these applications are highly sensitive to queuing delay, tail latency and short-term throughput fluctuations. Unlike traditional video-on-demand (VOD) which can tolerate jitter through pre-buffering, real-time services require transport protocols that can adapt quickly to dynamic network conditions without overfilling buffers or introducing the very delay spikes that degrade the quality of experience (QoE).

Traditional loss-based CCAs, such as TCP Reno [7] and the long-standing Linux default, cubic [8], rely on packet loss to infer congestion [9]. While effective for bulk data transfer, their core philosophy requires periodically filling network buffers to the point of overflow, this behaviour leads to a well-known problem called "Bufferbloat" [10]. To mitigate these issues, delay-based and hybrid CCAs like TCP-Lola [11], TCP-Vegas [12], Copa [13], Sprout [14, 15] and GCC [16] were proposed to control queuing delay more proactively. More recent efforts have focused on model-driven CCAs, such as PCC [17], which aim to strike a better balance between throughput and latency. However, many of these designs either converge slowly, misinterpret non-congestion-related delay signals, or require parameter tuning that undermines their performance across diverse network environments [18].

BBR (bottleneck bandwidth and round-trip propagation time) [19] congestion control represents a departure from traditional loss-based congestion control algorithms by explicitly modelling

This is an open access article under the terms of the Creative Commons Attribution License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2026 The Author(s). *IET Communications* published by John Wiley & Sons Ltd on behalf of The Institution of Engineering and Technology.

TABLE 1 | Comparison of BBR variants with respect to queue management and RTT handling.

Aspect	BBRv1	BBRv2	BBRv3
RTT reference	MinRTT (RTprop)	MinRTT with inflight bounds	MinRTT with refined inflight control
ProbeBW behaviour	Periodic aggressive probing	Moderated probing with loss signals	Refined probing to reduce queue buildup
Queue draining mechanism	Periodic ProbeRTT	Enhanced ProbeRTT	Adaptive ProbeRTT scheduling
Loss sensitivity	Low	Medium (explicit loss response)	Higher (improved loss handling)
Target queue awareness	None	Partial (inflight cap)	Partial (dynamic inflight tuning)

the bottleneck bandwidth (BtlBw) and the minimum round-trip propagation delay (RTprop). BBR aims to achieve high throughput while maintaining low queuing delay [20], outperforming Reno- and CUBIC-based schemes in many bandwidth-limited and shallow-buffer scenarios. BBRv2 [21] further refines this model by introducing stricter inflight control and more conservative probing behaviours to improve fairness and coexistence with loss-based flows [7, 22–24]. Despite these improvements, several studies have shown that BBRv2 still relies on periodic bandwidth probing and enforced ProbeRTT phases, which may introduce transient queue buildup and delay fluctuations, especially in low-latency or buffer-constrained environments [25–29]. BBRv3 [30, 31] builds on this by further refining pacing gains. Table 1 summarizes the key differences among BBRv1, v2 and v3 that are relevant to queue dynamics and RTT maintenance.

Despite these refinements, existing BBR variants continue to rely on periodic bandwidth probing as the fundamental mechanism for capacity discovery. Such probing may introduce transient queue buildup as a side effect. While the resulting delay may be tolerable in general-purpose scenarios, in real-time video streaming, such transient queues can cause delay spikes, which have been shown to correlate with playback stalls, rebuffering events and dropped frames [32, 33], ultimately degrading user experience [34].

To address these limitations, a number of BBR variants have been proposed with different optimization objectives. Fairness-oriented approaches, such as FaiRTT [35], incorporate RTT-aware pacing or inflight adjustments to mitigate the inherent RTT unfairness among competing flows. While these methods improve long-term throughput fairness, they primarily focus on equilibrium behaviour and provide limited control over short-term queue dynamics. Other works, including Ad-BBR [36] and BBR-R [37], aim to enhance delay stability by moderating probing aggressiveness or refining bandwidth gain parameters. However, they often rely on static thresholds or predefined probing patterns, which may not adapt well to rapidly changing network states.

Beyond end-host congestion control, several studies have investigated the cooperation between BBR and network-side active queue management (AQM) [38] mechanisms. By combining BBR or its variants with AQM schemes such as CoDel, PIE [39]. However, the deployment of AQM requires network infrastructure support, which may not always be available in practice.

More recently, learning-based and model-assisted approaches have been proposed to optimize BBR parameters, such as pacing rate or bandwidth estimation windows, using machine learning techniques [40, 41]. These methods leverage historical network observations to predict favourable control actions and have shown potential performance gains in specific scenarios [42]. Nevertheless, their increased complexity, training overhead and limited interpretability pose challenges for stable deployment in production environments [43]. BBR-S introduces a Kalman-filter-based estimation framework to smooth bandwidth and RTT measurements [44].This approach improves the reliability of BBR's state transitions in fast-varying environments, though it relies on carefully tuned model parameters.

In addition, to address BBR's sensitivity to heterogeneous loss characteristics in wireless networks, Yinker propose an adaptive pacing-gain adjustment strategy driven by observed loss rates and congestion levels [45]. However, such parameter-driven adaptations remain reactive to short-term measurements and may still suffer from queue oscillations in low-latency settings.

Beyond TCP-based BBR variants, recent transport protocols and hybrid congestion control algorithms have also been investigated to improve performance under diverse network conditions. QUIC [46], as a user-space transport protocol, decouples congestion control from the operating system kernel, enabling rapid deployment and experimentation of congestion control algorithms. Several QUIC implementations adopt BBR or BBR-derived controllers [47, 48], demonstrating comparable throughput performance while benefiting from reduced handshake latency and flexible control-plane evolution. However, QUIC itself does not fundamentally alter the congestion control logic and still inherits the limitations of the underlying BBR mechanism when operating in low-latency or shallow-buffer networks. In addition, BDP-Veno [49] represents a class of hybrid congestion control schemes that combine bandwidth-delay product estimation with delay and loss signals to distinguish between congestion-induced and random losses. By leveraging BDP-aware window adjustment, BDP-Veno improves throughput stability in wireless and heterogeneous environments. Nevertheless, such approaches primarily rely on end-to-end measurements and implicit queue inference, lacking explicit coordination with in-network queue dynamics.

In contrast to the above approaches, AQDC-BBR focuses on explicitly modeling residual queue dynamics at the sender side while preserving the original BBR state machine. By integrating

17518636, 2026, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/cmu2.70141, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

queue-aware decisions into existing probing phases, AQDC-BBR aims to reduce unnecessary queue accumulation without sacrificing bandwidth utilization, making it particularly suitable for low-latency and real-time transmission scenarios.

We implement the AQDC-BBR algorithm in both NS-3 [50] and Mininet [51], and the simulation and experimental results demonstrate that: 1) AQDC-BBR exhibits robustness across diverse network scenarios; 2) Compared with Copa and PCC, AQDC-BBR reduces tail latency by 26% and 25%, respectively; 3) Compared with BBR and its variants, it achieves over 25% reduction in tail latency while maintaining stable, high throughput.

Our main contributions are as follows:

- We identify and formalize the concept of "queue debt" in BBR-style probing, quantifying the residual queue buildup caused by periodic bandwidth overshoot.
- We propose AQDC-BBR, a novel and lightweight extension to BBR that actively tracks and compensates for this queue debt by adjusting the inflight data target, enabling accurate RTT tracking without sacrificing throughput.
- We implement AQDC-BBR in NS-3 and rstun, and conduct comprehensive simulations. Results show that AQDC-BBR reduces tail latency by 25%–30% compared with other schemes while maintaining consistently high throughput.

The remainder of this paper is organized as follows. Section 2 discusses the design motivation of AQDC-BBR. Section 3 describes its design and implementation in detail. Section 4 presents the simulation and experimental results, and finally, Section 5 summarizes the key findings of this paper.

2 | Background and Motivation

2.1 | BBR Congestion Control: Mechanism and Potential Limitations

BBR (bottleneck bandwidth and round-trip time) is a model-driven congestion control algorithm that represents a significant departure from traditional loss-based approaches. Its core idea is to build an explicit model of the network path by continuously estimating two key parameters: the maximum bottleneck bandwidth (BtlBw) and the minimum round-trip time (RTprop). The product of these two values serves as an estimate of the bandwidth-delay product (BDP), which represents the amount of data required to exactly fill the network pipe.

BBR's primary goal is to regulate its congestion window and sending rate to maintain the amount of inflight data at approximately one BDP. This allows the algorithm to achieve high throughput and low latency without relying on packet loss signals.

$$\text{inflight}_{\text{target}} = \text{maxBw} \cdot \text{minRTT} \tag{1}$$

BBR's operation process can be divided into four phases:

- Startup: Rapidly increases the sending rate to quickly discover the network's approximate capacity. The algorithm exits the StartUp phase when the newly calculated Bw_{max} does not increase by 25% for three consecutive RTTs.
- Drain: In this phase, the algorithm empty the queue created during StartUp. Once the inflight data is reduced to 1*BDP, the algorithm transitions to the ProbeBW phase.
- ProbeBW: The steady-state phase where BBR operates after discovering the initial bandwidth. During this phase, the sending rate is periodically adjusted according to a predefined pacing gain cycle, typically [1.25, 0.75, 1, 1, 1, 1, 1, 1] multiplied by the estimated bottleneck bandwidth Bw_{max}. Specifically, BBR first transmits at a rate of $1.25 \times \text{Bw}_{\text{max}}$ for one RTT to probe potential bandwidth increases. In the subsequent RTT, the pacing rate is reduced using a downward gain, with the intention of draining the excess data injected during the probing step. The remaining six RTTs operate at the nominal rate of $1 \times \text{Bw}_{\text{max}}$ to stabilize the delivery rate and maintain a consistent estimate of RT_{prop}.
- ProbeRTT: If no smaller RTT sample is observed within a fixed interval of 10 s, BBR enters the ProbeRTT phase to refresh RTprop. During this phase, the congestion window is temporarily reduced to 4 MSS for about 200 ms in order to drain the bottleneck queue and re-measure the minimum RTT. Afterward, the algorithm returns to the ProbeBW phase if the flow is not application-limited; otherwise, it switches to the StartUp phase.

In practice, however, this design introduces a critical side effect: the rate reduction during the ProbeDown phase is often insufficient to completely drain the queue generated during the ProbeUp phase.

Consider a connection with an estimated bottleneck bandwidth BtlBw = 100 Mbps and a minimum RTT $\text{RT}_{\text{prop}} = 20$ ms. During the ProbeUp sub-phase, the pacing rate is increased to 125 Mbps. Since the arrival rate at the bottleneck temporarily exceeds the service rate of the link, excess data is inevitably buffered. In this example, approximately $(125 - 100)$ Mbps $\times$ 20 ms = 0.5 Mb of data is injected into the bottleneck queue within one RT_{prop}. Following this aggressive probing, BBR enters the ProbeDown sub-phase and reduces the pacing rate using a downward gain. Applying a gain of 0.75 to the current pacing rate results in a reduced sending rate of approximately 94 Mbps. This leaves a rate headroom of only about 6 Mbps for draining the queue. Over one RTT, this headroom allows at most 6 Mbps $\times$ 20 ms = 0.125 Mb of queued data to be removed, which is insufficient to fully drain the queue accumulated during ProbeUp. As a result, a residual queue of roughly 0.375 Mb remains after each ProbeBW cycle.

In practice, this systematic imbalance leads to a periodic yet gradually increasing "sawtooth" pattern in both queue length and RTT, as illustrated in Figure 1. Over long-lived connections, the residual queue accumulates across successive cycles, causing the RTT observed by the sender to drift away from the true RT_{prop}. Consequently, key model parameters become contaminated until the queue is eventually cleared by the

17518636, 2026, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/cmu2.70141, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

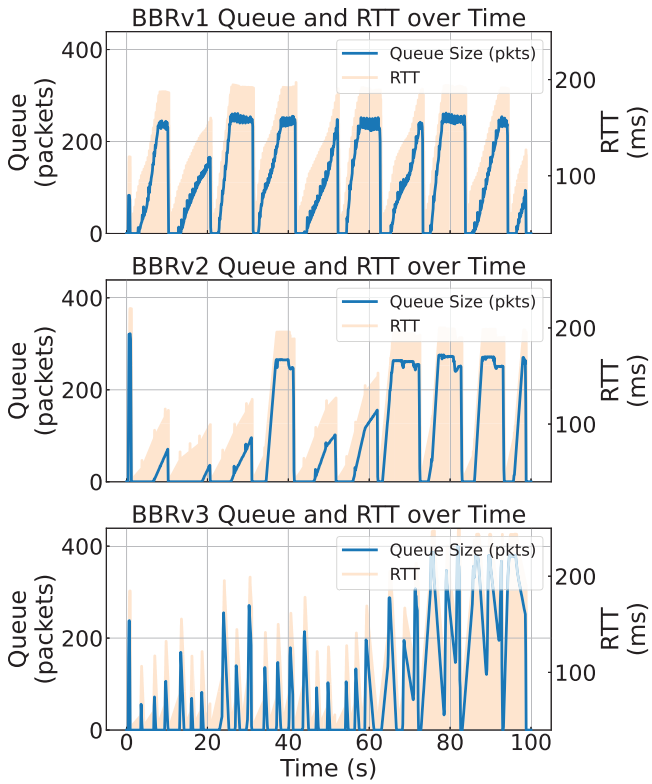


FIGURE 1 | Queue buildup and RTT evolution under BBR variants.

aggressive draining behaviour in the subsequent ProbeRTT phase.

2.2 | Design Challenges and Insights

Although BBR aims to achieve both high throughput and low latency, its active probing mechanism introduces an inherent tension in practice, which is particularly detrimental to latency-sensitive applications such as real-time video.

2.2.1 | Limitation 1: Fairness-Driven Bandwidth Probing Inevitably Creates and Sustains Queuing

To achieve multi-flow fairness and ensure effective competition with loss-based flows (such as CUBIC), BBR must periodically send data at a rate higher than its current bandwidth estimate to probe for potential available bandwidth. This excessive data injection is a necessary condition for fairness convergence and bandwidth discovery. However, when the extra data cannot be immediately absorbed by the network, the surplus inevitably accumulates in the bottleneck buffer, forming a queue and elevating the RTT.

At first glance, it may seem that the sender could gradually drain these queues by reducing the sending rate after the probing phase. In practice, however, this intuition does not hold under BBR's control framework. Multi-flow fairness in BBR is not achieved through explicit scheduling but emerges from dynamic competition within the buffer. Only when a certain queue is actually formed can each flow compete for its share of the buffer, eventually converging to a relatively fair allocation. Attempting to immediately clear the queues after probing would disrupt the previously established bandwidth shares and compromise the convergence of fairness.

Moreover, BBR's bandwidth estimation does not directly measure the physical capacity of the bottleneck link; it infers the currently achievable service rate from the rate of ACK returns. In multi-flow scenarios, this service rate is dynamically determined through buffer competition. When a flow sends at a rate higher than its current bandwidth estimate during the ProbeUP phase, the instantaneous arrival rate at the bottleneck temporarily exceeds the link's service rate, forcing excess packets into the buffer. This queuing not only increases RTT but also changes the flow's share of the buffer. A higher buffer share allows the flow to be scheduled more frequently, which is reflected in the ACK returns as an increased delivery rate and thus updates the bandwidth estimate.

In other words, the additional bandwidth discovered during ProbeUP does not correspond to idle physical capacity, but rather arises from occupying more buffer space and temporarily outcompeting other flows. Further, the queues formed during the probing phase contribute directly to the subsequent bandwidth estimation. After probing, BBR sets sending rates in the ProbeDown or steady phases based on the updated bandwidth estimate, which is obtained under the presence of existing queues. As a result, even if the sender reduces its rate in ProbeDown, the target rate typically remains close to the actual bottleneck capacity, leaving insufficient headroom to drain the existing queue. In networks without explicit queue feedback, these queues cannot naturally dissipate within the ProbeBW cycle, creating a structural residual: it is not a transient overshoot, but a natural outcome of the probe–occupy–estimate loop.

In summary, BBR faces an unavoidable structural tension: on one hand, bandwidth probing must allow queue formation to achieve stable fair convergence; on the other hand, the queues introduced by probing cannot be immediately drained without compromising fairness, resulting in persistently elevated RTTs.

This tension constitutes the first core challenge in BBR's design:

Challenge 1. How can the algorithm avoid long-term queue accumulation and RTT inflation caused by its own probing, without undermining bandwidth probing or multi-flow fairness?

2.2.2 | Limitation 2: ProbeRTT is a Brute-Force and Inefficient Reset

For the reasons discussed above, BBR introduces the ProbeRTT mechanism, which temporarily reduces the sending rate significantly, ensuring that the link's service capacity exceeds the injection rate. This forcibly clears residual queues in the bottleneck buffer and allows the RTT to be measured close to the true propagation delay. While ProbeRTT is effective at restoring near-propagation RTT, it comes at a substantial cost.

First, ProbeRTT applies a sudden and large reduction in sending rate. During this period, the sender almost completely

17518636, 2026, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/cmu2.70141, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

relinquishes its share of the bottleneck bandwidth, resulting in a noticeable drop in throughput. This abrupt rate interruption introduces significant throughput fluctuations, which are particularly detrimental to applications such as video streaming that rely on smooth and continuous delivery rates.

Second, the operation of ProbeRTT lacks specificity. Regardless of whether the queues accumulated during the previous probing phase are large or small, ProbeRTT applies the same reset-to-zero approach, without considering the cause or the actual scale of the queues. This history-agnostic and queue-size-agnostic design makes ProbeRTT a coarse-grained system reset rather than a fine-grained adjustment to the side effects of probing.

Moreover, the triggering and execution of ProbeRTT are not based on the current queue state, but instead rely on fixed intervals or the expiration of the minimum RTT measurement. As a result, its timing often fails to match the actual queue evolution: in most cases, it either intervenes too late or too aggressively, further amplifying performance volatility.

In summary, ProbeRTT does not mechanistically eliminate queues induced by bandwidth probing. Instead, it periodically and disruptively repairs the system after the fact. This design reflects BBR's trade-off between fairness convergence and low-latency objectives, rather than providing a fundamental resolution of the conflict.

This gives rise to the second design challenge:

Challenge 2. Can a more continuous and targeted compensation mechanism be designed, allowing the sender to adjust based on the actual queue size and thereby reduce reliance on ProbeRTT interventions?

2.2.3 | Design Insights

Based on the above analysis, three key insights can be extracted to guide the design of AQDC-BBR:

- Queue accumulation caused by bandwidth probing is an inherent byproduct of fairness and bandwidth discovery, rather than an anomaly.
- For each successful bandwidth probe, the amount of induced queue is quantitatively related to the bandwidth increment and RTprop.
- By explicitly tracking and managing the queues generated by its own probing, a sender can, in principle, mitigate long-term queuing effects without undermining probing or fairness mechanisms.

3 | AQDC-BBR Design

3.1 | Design Rationale: From Passive Reaction to Active Control

By explicitly separating bandwidth and RTT signals, BBR is able to achieve high throughput while largely avoiding persistent loss. However, its latency management mechanism remains fundamentally passive.

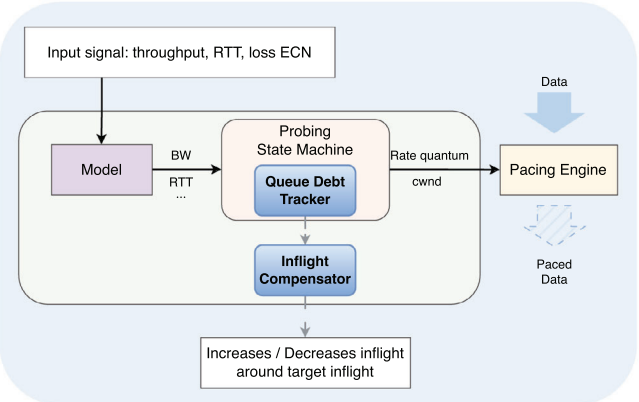


FIGURE 2 | AQDC-BBR control loop with queue debt compensation modules.

Specifically, BBR relies on the ProbeRTT state to periodically reset its delay baseline. ProbeRTT is only triggered after the network has already experienced sustained queue buildup, manifested as prolonged RTT inflation. Once triggered, the sender drastically reduces its sending rate to drain the queue and remeasure the minimum RTT. This stop-and-drain behaviour introduces two inherent limitations.

First, latency control is reactive rather than proactive. Queue buildup must already be significant before any corrective action is taken, resulting in persistent queues and elevated tail latency for long-lived flows. This behaviour is particularly harmful for delay-sensitive applications such as real-time video streaming, as confirmed by the RTT distributions shown in Figure 3.

Second, ProbeRTT introduces abrupt throughput disruption. The pre-scheduled and aggressive rate reduction during ProbeRTT temporarily relinquishes bottleneck bandwidth, causing noticeable throughput jitter. Such sharp oscillations undermine delivery smoothness and degrade application-level quality of experience (QoE).

At a deeper level, these limitations stem from a structural coupling between bandwidth probing and latency recovery in BBR. During the ProbeBW phase, BBR intentionally injects excess traffic to discover available bandwidth and enforce multi-flow fairness. However, the queues created by this probing are only cleared through infrequent and disruptive global resets. This observation motivates a fundamental design shift: bandwidth probing and latency control should be decoupled. Rather than relying on delayed, heavyweight resets, the sender should continuously and proactively manage the queue backlog induced by its own probing behaviour.

Based on this insight, we propose active queue debt compensation (AQDC), a lightweight control mechanism that operates alongside the BBR state machine (see Figure 2). AQDC continuously tracks and compensates for the excess inflight data introduced during bandwidth probing, enabling stable latency control through smooth, fine-grained adjustments instead of

17518636, 2026, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/cmu2.70141, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

periodic resets. This transition from passive reaction to active control forms the foundation of AQDC-BBR.

3.2 | AQDC Architecture Overview

AQDC is designed as a lightweight overlay framework that runs in parallel with the original BBR control logic. Its key design principle is non-intrusiveness: AQDC does not alter BBR's bandwidth estimation pipeline, state machine semantics, or pacing logic. As a result, the powerful bandwidth discovery and fairness properties of BBR are fully preserved.

As illustrated in Figure 2, AQDC consists of two tightly coupled components:

3.2.1 | Real-Time Queue Monitor

This module is responsible for tracking the queue debt induced by aggressive bandwidth probing. Instead of inferring queue occupancy from noisy RTT measurements, the monitor leverages BBR's internal model updates to quantify excess inflight data introduced during probing events. The tracked queue debt represents the amount of data that exceeds the true bandwidth-delay product and therefore contributes to buffer occupancy.

3.2.2 | Proportional Compensation Actuator

This module compensates the accumulated queue debt by dynamically adjusting the sender's inflight target. Rather than triggering discrete state transitions, the actuator applies smooth and proportional corrections based on the severity of the tracked debt. By gradually reducing excess inflight data, AQDC repays the queue debt while minimizing throughput disruption.

Together, these components form a closed-loop control layer that continuously suppresses transient queue growth caused by probing. Unlike ProbeRTT, AQDC operates on sub-RTT timescales and adapts to real-time queue dynamics. This hierarchical design allows AQDC-BBR to maintain high throughput under normal conditions while preventing sustained delay inflation, effectively bridging the long-standing gap between bandwidth discovery and latency control.

3.3 | Queue Debt Modelling

As BBR periodically increases pacing rate during ProbeBW, it may inject more data than the link can handle. When this increase is not fully absorbed by available bandwidth, packets accumulate in the queue. We formalize this phenomenon as queue debt, which quantifies the mismatch between aggressive probing and the network's actual service capability.

Consider a ProbeBW cycle where the estimated bottleneck bandwidth increases from bw_i to bw_{i+1} ($bw_{i+1} > bw_i$). This discovery implies that during the probing period (one minRTT), the sender injected more data than the previous bandwidth bw_i could handle, and this excess data was absorbed by the network buffer. The magnitude of this newly created queue, or the incremental queue debt B_i, can be quantified as:

$$B_i = (bw_{i+1} - bw_i) \cdot \text{minRTT} \tag{2}$$

The cumulative queue debt across multiple probing cycles is:

$$B_{\text{total}} = \sum_j B_j \tag{3}$$

This accumulated backlog acts as excess occupancy in the link buffers, causing persistent interference with the true network latency. Accordingly, the observed RTT grows with cumulative queue debt:

$$\begin{aligned} \text{RTT}_{\text{observed}} &= \text{minRTT} + \frac{B_{\text{total}}}{C} \\ &= \text{minRTT} + \frac{1}{C}\sum_{j=1}^{n}(bw_{j+1} - bw_j) \cdot \text{minRTT} \end{aligned} \tag{4}$$

Assuming each probing step increases bandwidth by a fraction α:

$$\text{RTT}_{\text{observed}} \approx \text{minRTT} + \frac{\text{minRTT} \cdot bw_0}{C}((1+\alpha)^n - 1) \tag{5}$$

where C denotes the bottleneck service rate.

As shown in Equation (5), RTT grows exponentially with each cycle if the sender continues probing without compensation. This is consistent with our experimental observations in Figure 1, where long-lived BBR flows gradually accumulate queue even in the absence of competing traffic. AQDC-BBR is designed to counteract this by explicitly tracking B_{total}.

The presence of queue debt also impacts throughput. As RTT inflates, maintaining throughput requires a larger inflight window, reinforcing the delay-throughput tradeoff:

The effective throughput under accumulated queue debt is:

$$\text{Throughput} \approx \min\left(\text{BDP}, \frac{\text{inflight}}{RTT_{\text{observed}}}\right) \tag{6}$$

These observations highlight the necessity of explicitly tracking and managing queue debt. AQDC-BBR addresses this challenge by continuously estimating B_{total} and using it as a first-class control signal.

3.4 | Queue Debt Compensation Logic and Implementation

AQDC replaces the timer-based semantics of ProbeRTT with a debt-driven, goal-oriented control mechanism. Instead of enforcing a fixed duration, AQDC defines a precise and dynamic exit condition based on the tracked queue debt.

Specifically, AQDC continuously monitors the sender's inflight data during ProbeRTT and terminates the state as soon as the accumulated queue debt has been fully repaid. The exit condition

17518636, 2026, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/cmu2.70141, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

ALGORITHM 1 | Queue debt compensation in BBR.

```
State Variables: bw_prev ← 0, queue_debt_total ← 0
function UpdateBwEstimate bw_new
    if bw_new > bw_prev then
        ΔB ← (bw_new − bw_prev) · minRTT
        queue_debt_total ← queue_debt_total + ΔB
    end if
    bw_prev ← bw_new
end Function
function SetInflightTarget()
    BDP ← bw_current · minRTT
    inflight_target ← BDP − queue_debt_total
end Function
function OnProbeRTTStart()
                                        ▷ Drain excess queue
    Call SetInflightTarget
end Function
function OnProbeRTTEnd()
    queue_debt_total ← 0
end Function
```

is defined as:

$$\text{Inflight}_{\text{target}} \leq \text{BDP} - B_{\text{total}} \tag{7}$$

Here, $\text{BDP} - B_{\text{total}}$ represents the exact amount of data that should remain in flight once all probe-induced queues have been drained. Reaching this threshold (subject to a minimum floor (e.g. 4 MSS)) indicates that the bottleneck buffer is effectively empty and that the observed RTT has converged back to the true propagation delay.

Once this condition is satisfied, AQDC immediately exits ProbeRTT and resumes normal ProbeBW operation. No fixed timers, conservative lower bounds, or heuristic delays are involved. As a result, the duration of ProbeRTT becomes fully adaptive, determined solely by the magnitude of accumulated queue debt.

This compensation logic is integrated into the BBR state machine as detailed in Algorithm 1.

AQDC integrates into the BBR state machine with minimal modification, forming a closed-loop control process:

Accumulation (UpdateBwEstimate): During ProbeBW, whenever a new bandwidth estimate exceeds the previous one, AQDC computes the incremental queue debt and accumulates it into B_{total}.

Compensation (HandleProbeRTT): Upon entering ProbeRTT, AQDC disables the fixed-duration timer and continuously evaluates the dynamic exit condition based on inflight and queue debt.

Reset: Once ProbeRTT exits, B_{total} is reset to zero, preparing the controller for the next probing cycle.

This integration requires only a small number of additional state variables and minor changes to existing transition logic (approximately 30 lines of code in our implementation). Importantly, AQDC preserves BBR's original model and state semantics while transforming ProbeRTT into a precise, self-terminating queue compensation phase.

Although BBRv2 and BBRv3 employ more complex and conservative probing mechanisms than BBRv1, AQDC's queue debt compensation logic can be applied seamlessly and identically across all versions. AQDC's universality stems from the fact that it focuses on the outcome of bandwidth probing (i.e. the increase in the bandwidth estimate) rather than the process (i.e. the specific pacing gain used). While BBRv1 uses an aggressive gain (e.g. 1.25) and BBRv2/v3 use more conservative values, the creation of queue debt (ΔB) is a direct consequence of any successful probe where $bw_{\text{new}} > bw_{\text{prev}}$. AQDC's debt accumulation module only cares about this bandwidth differential, making it agnostic to the underlying probing strategy. Therefore, integrating AQDC into BBRv2 and BBRv3 is identical to the BBRv1 implementation. We use the same code hooks to accumulate debt and the same goal-driven ProbeRTT mechanism to repay it. AQDC acts as an orthogonal, lightweight plugin that works effectively without needing to understand the complex hybrid-signal models of BBRv2/v3.

4 | Evaluation

4.1 | Methodology and Experimental Setup

We evaluate the proposed AQDC-BBR mechanism through extensive simulations and real-world experiments, focusing on its ability to control queue buildup, stabilize RTT and deliver low-latency performance in video streaming. We compare AQDC-BBR against BBRv1, BBRv2 and representative congestion control algorithms including cubic, PCC-Vivace and Copa, demonstrating its effectiveness across diverse network conditions. To ensure both protocol-level precision and application-level relevance, we adopt a dual-platform evaluation framework combining NS-3 simulations with Mininet-based video streaming experiments.

4.1.1 | NS-3 Network Simulations

NS-3 allows precise control over network parameters, including link bandwidth, propagation delay and buffer size. We deploy a dumbbell topology with a bottleneck link of 20 Mbps and peripheral links of 100 Mbps. Many scenarios are considered to evaluate fairness and queue dynamics. Our baselines include BBR v1, v2, v3, cubic and their AQDC-enhanced variants.

4.1.2 | Mininet Video Streaming Experiments

To complement packet-level simulations with application-level performance, we implement QUIC-based video streaming in Mininet using a Rust QUIC stack and ZLMediaKit as the video

17518636, 2026, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/cmu2.70141, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

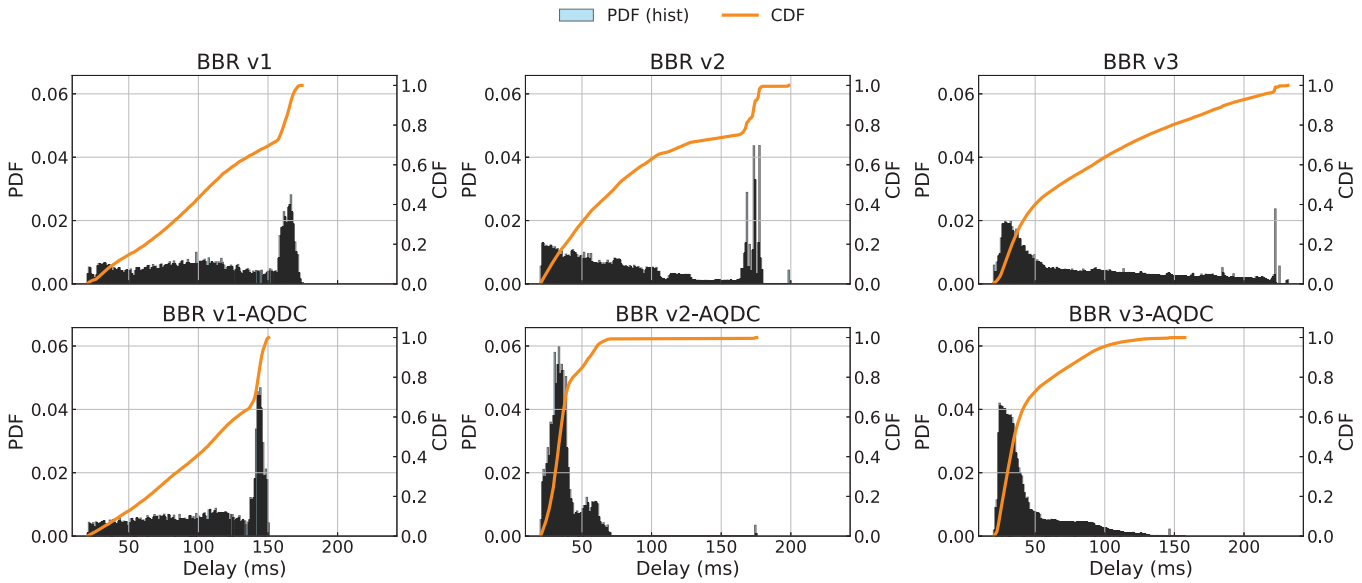


FIGURE 3 | RTT distribution (CDF and PDF) for BBR and AQDC-BBR variants.

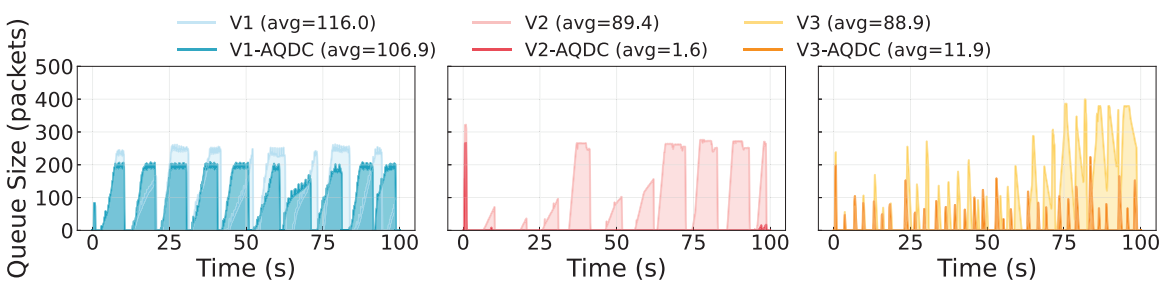


FIGURE 4 | Queue occupancy comparison between original and AQDC-enhanced BBR variants.

server. The same set of congestion control algorithms: BBR variants, AQDC-BBR, PCC and Copa are evaluated.

4.2 | NS-3 Simulation Results

4.2.1 | Scenario 1: Tail Latency and RTT Distributions

We evaluate AQDC-BBR in single-flow scenarios to investigate its impact on latency characteristics across different BBR versions. Specifically, we measure the RTT distributions for BBRv1, BBRv2 and BBRv3, along with their AQDC-enhanced variants. Each flow runs over a fixed-capacity bottleneck with controlled buffer settings, allowing us to isolate protocol-level behaviours. Figure 3 presents both the CDF and PDF of RTT measurements. The results show that AQDC-BBR consistently reduces tail latency across all versions, with 99th percentile RTT improvements ranging from 25% to 40% compared to their unmodified counterparts. This effect is particularly prominent in BBRv2 and BBRv3, where the original design tends to accumulate residual queues during repeated probing cycles.

While these statistical plots quantify the improvements in RTT, visualizing the queue size's temporal behaviour provides direct evidence of the underlying cause. Figure 4 illustrates the instantaneous queue size, contrasting the original BBR versions (light-coloured fills) against their AQDC-enhanced counterparts (darker fills).

The figure reveals a fundamental behavioural flaw in standard BBR: a pattern of cyclical, large-scale queue buildups. For all baseline versions, the queue periodically spikes from a near-zero level to peaks exceeding 200–250 packets. This oscillation is not random; it is a direct artefact of BBR's ProbeBW gain-cycling mechanism, where the algorithm intentionally sends faster than the link capacity to search for bandwidth. This recurring queue accumulation is precisely what inflates the RTT, making it the primary source of the high tail latency captured in our RTT CDF/PDF analysis.

In stark contrast, AQDC fundamentally alters this behaviour. The darker plots for AQDC-BBR show a remarkably stable and minimal queue, hovering consistently near zero. AQDC achieves this by preemptively curbing the aggressive over-sending during the probing phase, effectively preventing the queue from forming in the first place.

Notably, the magnitude of this improvement is most pronounced for BBRv2 and BBRv3. While AQDC-BBRv1 shows a tangible reduction in queue volatility, its AQDC-enhanced counterparts

17518636, 2026, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/cmu2.70141, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

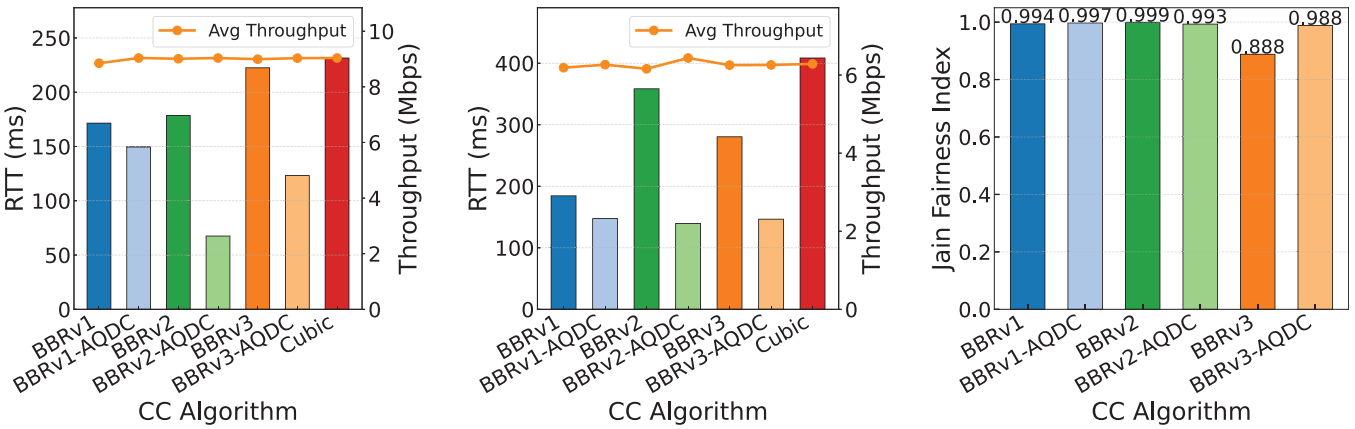


FIGURE 5 | NS-3 simulation-based evaluation of AQDC-BBR.

for v2 and v3 maintain a near-perfectly flat, zero-queue profile. We attribute this difference to the interplay between our current AQDC repayment logic and the intrinsic aggressiveness of each BBR version. BBRv1's original gain cycle is notoriously aggressive, creating a substantial "queue debt". The current one-to-one repayment strategy in AQDC, while beneficial, struggles to fully neutralize BBRv1's strong tendency to queue. Conversely, the more refined probing mechanisms in BBRv2 and BBRv3 generate smaller, more manageable queue oscillations, which our AQDC logic can perfectly counteract. This demonstrates that the effectiveness of AQDC is coupled with the baseline stability of the underlying BBR version.

4.2.2 | Scenario 2: Impact of Deep Buffers

To challenge AQDC-BBR in a more demanding environment, We evaluate the performance of AQDC-BBR under deep-buffered conditions, a common scenario in real-world deployments where buffer sizes at bottleneck routers exceed the bandwidth-delay product. In this experiment, we set the bottleneck queue to $4 \times$ BDP and observe how different congestion control variants behave in terms of both delay and throughput.

The results, presented in Figure 5a, are stark and unequivocally expose the classic Bufferbloat problem inherent in standard BBR. When confronted with an oversized buffer, the baseline BBR versions systematically fill it. Their 99th percentile RTT reaches an astonishing 230 ms, a value 4 times the network's base propagation delay. While maintaining high throughput, this extreme level of self-inflicted latency renders these versions entirely unsuitable for interactive or latency-sensitive applications.

In sharp contrast, the AQDC-enhanced versions exhibit remarkable resilience to this network condition. The P99 RTT for AQDC-BBR is a mere 150 ms, a figure that is not only dramatically lower than its standard counterpart but also nearly identical to its performance in a tightly-buffered ($1 \times$ BDP) environment. This consistency is critical. Crucially, this latency reduction is achieved with a negligible impact on throughput, which remains within 0.5% of the standard BBR's performance.

This outcome validates the core design principle and robustness of our queue compensation mechanism. Unlike standard BBR, which passively fills whatever buffer is available, AQDC actively manages the volume of inflight data to ensure the network queue is consistently kept at a minimal level, regardless of the physical buffer's actual size. By doing so, AQDC effectively decouples the protocol's behaviour from the bottleneck buffer depth, immunizing the flow against the detrimental effects of deep buffers.

4.2.3 | Scenario 3: Dynamic Bottleneck Scenario

To assess the adaptability and fairness of AQDC-BBR under realistic and challenging conditions, we designed a dynamic bottleneck scenario. Here, we evaluate two competing flows sharing a bottleneck link where the capacity alternates between 20 and 10 Mbps, mimicking the fluctuating conditions of mobile or congested networks.

A high-level statistical view in Figure 5b first reveals the outcome: standard BBR variants respond slowly to bandwidth changes, leading to significantly prolonged P99 RTT inflation. In contrast, AQDC-BBR consistently maintains low tail latency, demonstrating its superior responsiveness.

To understand the root cause of this poor statistical performance, we examine the instantaneous protocol behaviour. The throughput plot in Figure 6 highlights how the latency issue translates into severe performance degradation. While BBRv1 maintains relative fairness, it suffers from periodic throughput dips due to its ProbeRTT phase. This problem escalates dramatically in BBRv2 and BBRv3. Following the first bandwidth drop at 10 s, their fairness collapses, with one flow capturing nearly all the available capacity and effectively starving the other. In stark contrast, all AQDC-enhanced versions maintain excellent and stable fairness.

The reason for this chaotic throughput and fairness collapse is vividly illustrated in the corresponding instantaneous RTT plot (Figure 7). At the precise moment the bandwidth drops, the standard BBR variants experience a massive RTT spike, reaching upwards of 200 ms. This is the direct consequence of BBR carrying a large inflight volume sized for the previous 20 Mbps capacity. When the link is suddenly halved, this massive data volume overshoots the new BDP, creating a large, persistent

17518636, 2026, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/cmu2.70141, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

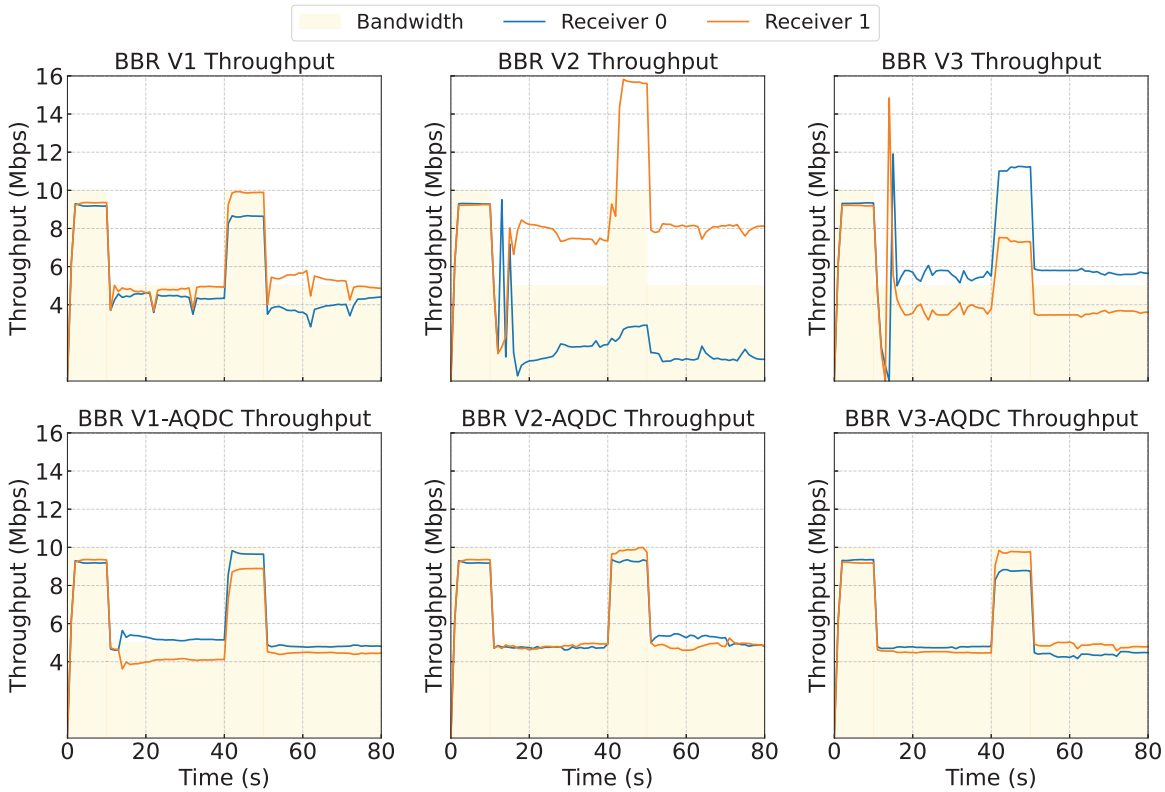


FIGURE 6 | Throughput fairness and adaptability under a dynamic bottleneck.

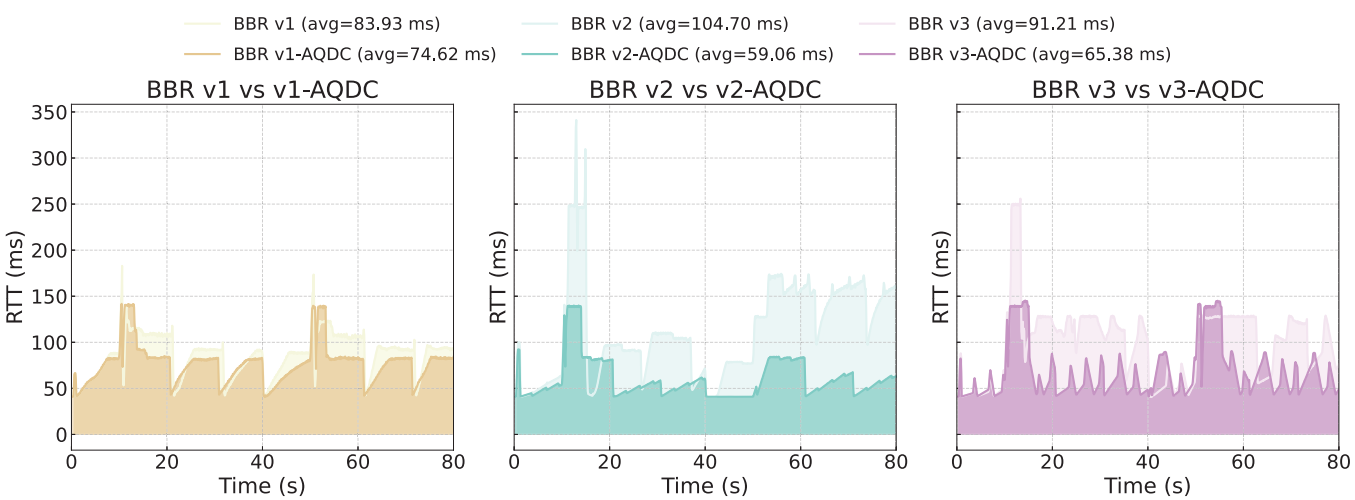


FIGURE 7 | RTT response to abrupt bandwidth changes.

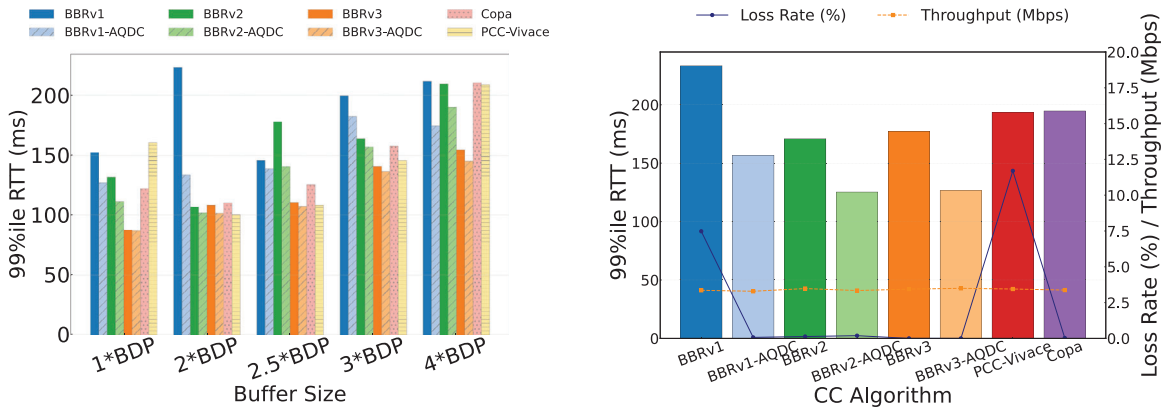
queue that triggers the high RTT and chaotic recovery seen in the throughput graph.

The RTT plot confirms that AQDC variants exhibit only a minor, transient bump in RTT, which quickly settles back to the baseline. Because AQDC proactively manages inflight to keep the queue minimal, its data volume is always much closer to the true, current BDP. Consequently, when the bandwidth drops, the overshoot is negligible, allowing the protocol to adapt gracefully. This demonstrates that AQDC's queue-aware design is the key to ensuring low latency, robust fairness and high stability in dynamic, multi-flow network environments.

4.2.4 | Scenario 4: Steady-State Multi-Flow Fairness

Having observed the dynamic fairness breakdown in Scenario 3, we now shift our focus to quantitatively evaluate the long-term fairness convergence in a stable, multi-flow environment. In this scenario, we measure the equilibrium state when several flows

17518636, 2026, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/cmu2.70141, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License



(a) Effect of Buffering on Tail Latency

(b) P99 RTT and Packet Loss

FIGURE 8 | Mininet simulation-based evaluation of AQDC-BBR.

share a bottleneck link with a constant capacity, using the widely-recognized Jain's Fairness Index [52] as the definitive metric.

The results are summarized in Figure 5c. The analysis reveals a known weakness in BBRv3, which exhibits a relatively poor fairness index in these crowded conditions. Notably, AQDC-BBRv3 improves this fairness index by a significant 11%.

This result is significant, demonstrating that applying AQDC does not create a trade-off with fairness but can instead actively remedy issues present in the baseline protocol. The improvement stems from AQDC's core mechanism: by preventing the formation of a standing queue, it eliminates the buffer as a point of contention. Aggressive flows are unable to "hog" the buffer space, which provides a more stable and equitable environment for all flows to converge to their fair share. This confirms that AQDC's explicit queue management not only dramatically reduces latency but also promotes a more equitable bandwidth distribution among competing flows.

4.3 | Video Streaming Scenario Results

4.3.1 | Scenario 5: Robustness to BDP Variation in Video Streaming

To evaluate the real-world deployability of AQDC-BBR, we assess its performance in a video streaming scenario across a wide spectrum of network conditions. This test is critical for evaluating quality of experience (QoE), where low and stable latency is paramount to prevent rebuffering events. We vary the network's capacity-delay product and buffering from extremely shallow (0.1 × BDP) to very deep (4 × BDP), simulating diverse environments from constrained mobile networks to over-buffered residential connections.

The 99th percentile RTT results, reported in Figure 8a, demonstrate that AQDC-BBR exhibits superior and remarkably consistent performance across all tested regimes. It consistently achieves over a 25% reduction in tail latency compared to its unmodified BBR counterparts while maintaining comparable throughput. Furthermore, it significantly outperforms other state-of-the-art algorithms, reducing P99 RTT by 26% compared to Copa and 25% compared to PCC-Vivace.

The key insight from this experiment is AQDC's fundamental robustness. Delay-based algorithms like Copa can be sensitive to the configured target delay, which may be mismatched for networks with very low or very high BDPs. Similarly, utility-based algorithms like PCC can struggle to converge optimally under rapidly changing or extreme network conditions. In contrast, AQDC's strength lies in its direct approach: it does not infer congestion from a delay target or a utility function; it directly manages the queue size.

This mechanism makes its behaviour fundamentally independent of the absolute BDP value or the physical buffer's depth. Whether the buffer is shallow or deep, AQDC's objective remains the same: keep the queue minimal. This robustness is achieved without any parameter tuning, highlighting its suitability for deployment in heterogeneous, "zero-configuration" environments where network characteristics are unknown and constantly changing.

4.3.2 | Scenario 6: End-to-End Performance in a Realistic Video Streaming Environment

To validate our findings in a holistic, application-level setting, we constructed a realistic video streaming testbed using Mininet for network emulation and ZLMediaKit, a high-performance streaming media server. This setup moves beyond abstract TCP flows to evaluate the tangible impact of the congestion control algorithm on a complete, end-to-end video delivery pipeline, which includes the server's own application-level pacing and logic.

The results, presented in Figure 8b, focus on two metrics that are paramount for video quality of experience (QoE): 99th percentile RTT and packet loss rate. AQDC-BBR demonstrates a decisive advantage on both fronts. It achieves a P99 RTT reduction of at least 25% compared not only to standard BBR but also to other state-of-the-art algorithms like PCC and Copa.

17518636, 2026, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/cmu2.70141, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Crucially, this dramatic reduction in latency is complemented by an improvement in reliability. The figure shows that AQDC-BBR maintains comparable or slightly lower packet loss rates. This is a direct consequence of its core design. Standard BBR, by operating with a larger inflight volume, keeps the bottleneck buffer consistently close to full, making it highly vulnerable to overflow and packet loss during even minor network fluctuations. In contrast, AQDC's proactive queue management provides a significant "safety margin," keeping the buffer nearly empty and thus making packet loss a rare event.

For a video streaming application, these dual benefits are critical. Lower tail latency directly translates to a lower risk of playback stalls and rebuffering events. Simultaneously, a lower packet loss rate ensures a more stable and reliable data stream, allowing the video player's adaptive bitrate (ABR) algorithm to confidently select higher-quality video segments.

5 | Conclusion

Model-based congestion control protocols like BBR face a fundamental dilemma: the aggressive bandwidth probing required to achieve high throughput inevitably creates queuing delay, which harms latency-sensitive applications. In this paper, we addressed this challenge by proposing active queue debt compensation (AQDC-BBR), a lightweight extension that reframes queue management as an explicit, proactive process. Our core insight is to model the excess queue incurred during ProbeBW as a debt and then systematically repay it by temporarily reducing the inflight data target, thereby preventing queue accumulation without resorting to the disruptive, periodic rate cuts of the standard ProbeRTT phase.

Our extensive evaluations in both NS-3 and a realistic Mininet-based video streaming testbed demonstrated the effectiveness of this approach across a wide range of challenging scenarios. We showed that AQDC-BBR consistently reduces P99 RTT by 25%–40% in single-flow, multi-flow and deep-buffer conditions. Crucially, it resolves the severe fairness collapse exhibited by modern BBR variants under dynamic bandwidth and maintains robust, low-latency performance in end-to-end video streaming applications.

6 | Future Work

Our future work will extend AQDC's foundation in three key directions. First, building on our preliminary work porting AQDC to an rstun-based QUIC implementation, we will conduct a comprehensive evaluation of its performance in complex, real-world QUIC traffic scenarios. Second, we plan to explore a data-driven approach by using a lightweight, online-trained neural network to learn an optimal debt repayment policy, moving beyond the current model-based heuristic to adapt to more subtle network dynamics. Finally, we envision augmenting BBR's core principles with compact, specialized AI models to create a new class of inference-augmented congestion control, making it more adaptive to unmodelled network behaviours.

Author Contributions

Ziyi Li: data curation, formal analysis, methodology, writing – original draft. **Jiao Zhang:** project administration, writing – review and editing. **Yuqing Wang:** supervision. **Peisheng Guo:** software, writing – review and editing. **Jiafeng Jiang:** validation, visualization.

Funding

The authors have nothing to report.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

References

1. X. Xie and X. Zhang, "Poi360: Panoramic Mobile Video Telephony Over LTE Cellular Networks," in *Proceedings of the 13th International Conference on Emerging Networking EXperiments and Technologies* (Association for Computing Machinery, 2017), 336–349.

2. H. Wang, T. Li, R. Shea, et al., "Toward Cloud-Based Distributed Interactive Applications: Measurement, Modeling, and Analysis," *IEEE/ACM Transactions on Networking* 26, no. 1 (2018): 3–16.

3. H. Chen, X. Zhang, Y. Xu, et al., "T-Gaming: A Cost-Efficient Cloud Gaming System at Scale," *IEEE Transactions on Parallel and Distributed Systems* 30, no. 12 (2019): 2849–2865.

4. Y. Guan, C. Zheng, X. Zhang, Z. Guo, and J. Jiang, "PANO: Optimizing 360 Video Streaming With a Better Understanding of Quality Perception," in *Proceedings of the ACM Special Interest Group on Data Communication* (Association for Computing Machinery, 2019), 394–407.

5. F. Qian, B. Han, Q. Xiao, and V. Gopalakrishnan, "Flare: Practical Viewport-Adaptive 360-degree Video Streaming for Mobile Devices," in *Proceedings of the 24th Annual International Conference on Mobile Computing and Networking* (Association for Computing Machinery, 2018), 99–114.

6. L. Lin, X. Liao, G. Tan, et al., "LiveRender: A Cloud Gaming System Based on Compressed Graphics Streaming," in *Proceedings of the 22nd ACM International Conference on Multimedia* (Association for Computing Machinery, 2014), 347–356.

7. M. Allman, V. Paxson, and E. Blanton, *TCP Congestion Control* (ICIR, 2009).

8. S. Ha, I. Rhee, and L. Xu, "Cubic: a new tcp-friendly high-speed tcp variant," *ACM SIGOPS Operating Systems Review* 42, no. 5 (July 2008): 64–74, https://doi.org/10.1145/1400097.1400105.

9. V. Jacobson, "Congestion Avoidance and Control," *ACM SIGCOMM Computer Communication Review* 18, no. 4 (1988): 314–329, https://doi.org/10.1145/52325.52356.

10. J. Gettys and K. Nichols, "Bufferbloat: Dark Buffers in the Internet: Networks Without Effective AQM May Again be Vulnerable to Congestion Collapse," *Queue* 9, no. 11 (2011): 40–54.

11. M. Hock, F. Neumeister, M. Zitterbart, and R. Bless, "TCP LoLa: Congestion Control for Low Latencies and High Throughput," in *2017 IEEE 42nd Conference on Local Computer Networks (LCN)* (IEEE, 2017), 215–218.

12. L. S. Brakmo, S. W. O'malley, and L. L. Peterson, "TCP Vegas: New Techniques for Congestion Detection and Avoidance," in

17518636, 2026, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/cmu2.70141, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

Proceedings of the Conference on Communications Architectures, Protocols and Applications (ACM, 1994), 24–35.

13. V. Arun and H. Balakrishnan, "Copa: Practical {Delay-Based} Congestion Control for the Internet," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)* (ACM, 2018), 329–342.

14. K. Winstein, A. Sivaraman, and H. Balakrishnan, "Stochastic Forecasts Achieve High Throughput and Low Delay Over Cellular Networks," in *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)* (ACM, 2013), 459–471.

15. S. Fouladi, J. Emmons, E. Orbay, C. Wu, R. S. Wahby, and K. Winstein, "Salsify:{Low-Latency} Network Video Through Tighter Integration Between a Video Codec and a Transport Protocol," in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI 18)* (ACM, 2018), 267–282.

16. G. Carlucci, L. De Cicco, S. Holmer, and S. Mascolo, "Congestion Control for Web Real-Time Communication," *IEEE/ACM Transactions on Networking* 25, no. 5 (2017): 2629–2642.

17. M. Dong, Q. Li, D. Zarchy, P. B. Godfrey, and M. Schapira, "{PCC}: Re-Architecting Congestion Control for Consistent High Performance," in *12th USENIX Symposium on Networked Systems Design and Implementation (NSDI 15)* (ACM, 2015), 395–408.

18. P. Goyal, A. Narayan, F. Cangialosi, S. Narayana, M. Alizadeh, and H. Balakrishnan, "Elasticity Detection: A Building Block for Internet Congestion Control," *arXiv:1802.08730* (2018).

19. N. Cardwell, Y. Cheng, C. S. Gunn, S. H. Yeganeh, and V. Jacobson, "BBR: Congestion-Based Congestion Control: Measuring Bottleneck Bandwidth and Round-Trip Propagation Time," *Queue* 14, no. 5 (2016): 20–53.

20. L. Kleinrock, "Power and Deterministic Rules of Thumb for Probabilistic Problems in Computer Communications," in *ICC 1979; International Conference on Communications* (NASA, 1979), 43.1.1–43.1.10.

21. N. Cardwell, Y. Cheng, S. H. Yeganeh, et al., "BBR v2: A Model-Based Congestion Control IETF 105 Update," *Presentation at IETF105* (IETF, 2019), 1–31.

22. J. Gomez, E. Kfoury, J. Crichigno, E. Bou-Harb, and G. Srivastava, "A Performance Evaluation of TCP BBRv2 Alpha," in *2020 43rd International Conference on Telecommunications and Signal Processing (TSP)* (IEEE, 2020), 309–312.

23. E. F. Kfoury, J. Gomez, J. Crichigno, and E. Bou-Harb, "An Emulation-Based Evaluation of TCP BBRv2 Alpha for Wired Broadband," *Computer Communications* 161 (2020): 212–224.

24. P. Hurtig, H. Haile, K.-J. Grinnemo, et al., "Impact of TCP BBR on Cubic Traffic: A Mixed Workload Evaluation," in *2018 30th International Teletraffic Congress (ITC 30)* (IEEE, 2018), 218–226.

25. D. Scholz, B. Jaeger, L. Schwaighofer, D. Raumer, F. Geyer, and G. Carle, "Towards a Deeper Understanding of TCP BBR Congestion Control," in *2018 IFIP Networking Conference (IFIP Networking) and Workshops* (IEEE, 2018), 1–9.

26. M. Hock, R. Bless, and M. Zitterbart, "Experimental Evaluation of BBR Congestion Control," in *2017 IEEE 25th International Conference on Network Protocols (ICNP)* (IEEE, 2017), 1–10.

27. Y. Cao, A. Jain, K. Sharma, A. Balasubramanian, and A. Gandhi, "When to Use and When Not to Use BBR: An Empirical Analysis and Evaluation Study," in *Proceedings of the Internet Measurement Conference* (ACM, 2019), 130–136.

28. F. Yang, Q. Wu, Z. Li, Y. Liu, G. Pau, and G. Xie, "BBRv2+: Towards Balancing Aggressiveness and Fairness With Delay-Based Bandwidth Probing," *Computer Networks* 206 (2022): 108789.

29. M. Alizadeh, A. Greenberg, D. A. Maltz, et al., "Data Center TCP (DCTCP)," in *Proceedings of the ACM SIGCOMM 2010 Conference* (ACM, 2010), 63–74.

30. N. Cardwell, Y. Cheng, K. Yang, et al., "BBRv3: Algorithm Bug Fixes and Public Internet Deployment," *Proceedings of the IETF-117* 26 (ACM, 2023), 97–99.

31. N. Cardwell, Y. Cheng, K. Yang, et al., *BBR Congestion Control: Fundamentals and Updates* (Stanford University, 2023).

32. Z. Meng, Y. Guo, C. Sun, et al., "Achieving Consistent Low Latency for Wireless Real-Time Communications With the Shortest Control Loop," in *Proceedings of the ACM SIGCOMM 2022 Conference* (ACM, 2022), 193–206.

33. T. Kämäräinen, M. Siekkinen, A. Ylä-Jääski, W. Zhang, and P. Hui, "A Measurement Study on Achieving Imperceptible Latency in Mobile Cloud Gaming," in *Proceedings of the 8th ACM on Multimedia Systems Conference* (ACM, 2017), 88–99.

34. S. Cheng, H. Hu, X. Zhang, and Z. Guo, "Rebuffering But Not Suffering: Exploring Continuous-Time Quantitative QoE by User's Exiting Behaviors," in *IEEE INFOCOM 2023-IEEE Conference on Computer Communications* (IEEE, 2023), 1–10.

35. A. Abrol, P. M. Mohan, and T. Truong-Huu, "FaiRTT: An Empirical Approach for Enhanced RTT Fairness and Bottleneck Throughput in BBR," in *2024 IEEE International Conference on Communications Workshops (ICC Workshops)* (IEEE, 2024), 673–678.

36. M. Wang, X. Zhang, F. Jing, and M. Gao, "Ad-BBR: Enhancing Round-Trip Time Fairness and Transmission Stability in TCP-BBR," *Future Internet* 17, no. 5 (2025): 189.

37. S. Zheng, J. Liu, X. Yan, Z. Xing, X. Di, and H. Qi, "BBR-R: Improving BBR Performance in Multi-Flow Competition Scenarios," *Computer Networks* 254 (2024): 110816.

38. K. Nichols and V. Jacobson, "Controlling Queue Delay," *Communications of the ACM* 55, no. 7 (2012): 42–50.

39. J. M. Voas, "PIE: A Dynamic Failure-Based Technique," *IEEE Transactions on Software Engineering* 18, no. 8 (1992): 717.

40. M. Ahsan, A. Ahmad, R. Akhtar, and I. Javed, "Modeling BBR v3 Congestion Control Behavior Using Supervised ML Techniques," *Journal of Computing & Biomedical Informatics* 9, no. 2 (2025): 1–9.

41. V. Mhaske, K. Jain, S. K. Thatikonda, and A. Kunwar, "Improvement of Inter-Protocol Fairness for BBR Congestion Control Using Machine Learning," *arXiv:2312.11790* (2023).

42. Z. Tafa and V. Milutinovic, "Machine Learning in Congestion Control: A Survey on Selected Algorithms and a New Roadmap to Their Implementation," *arXiv:2112.15522* (2021).

43. M. Mazilu, L. Giacomoni, and G. Parisis, "Learning-Based Vs Human-Derived Congestion Control: An In-Depth Experimental Study," *arXiv:2510.25105* (2025).

44. F. Chiariotti, A. Zanella, S. Kucera, and H. Claussen, "BBR-S: A Low-Latency BBR Modification for Fast-Varying Connections," *IEEE Access* 9 (2021): 76364–76378.

45. Y. Xie, X. Jiang, G. Gong, Z. Jiang, G. Jin, and H. Chen, "Yinker: A Flexible BBR to Achieve the High-Throughput and Low-Latency Data Transmission over Wi-Fi and 5G Networks," *Computer Networks* 222 (2023): 109530.

46. A. Langley, A. Riddoch, A. Wilk, et al., "The QUIC Transport Protocol: Design and Internet-Scale Deployment," in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication* (ACM, 2017), 183–196.

47. H. Haile, K.-J. Grinnemo, S. Ferlin, P. Hurtig, and A. Brunstrom, "Performance of QUIC Congestion Control Algorithms in 5G Networks," in *Proceedings of the ACM SIGCOMM Workshop on 5G and Beyond Network Measurements, Modeling, and Use Cases* (ACM, 2022), 15–21.

48. Z. Zhang, S. Li, Y. Ge, G. Xiong, Y. Zhang, and K. Xiong, "PBQ-Enhanced QUIC: QUIC With Deep Reinforcement Learning Congestion Control Mechanism," *Entropy* 25, no. 2 (2023): 294.

17518636, 2026, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/cmu2.70141, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License

49. S. P. Biswal and S. Patel, "A Bandwidth Delay Product Based Modified Veno for High-Speed Networks: BDP-Veno," *Journal of Network and Computer Applications* 231 (2024): 103983.

50. G. F. Riley and T. R. Henderson, "The NS-3 Network Simulator," in *Modeling and Tools for Network Simulation* (Springer, 2010), 15–34.

51. Z. Xiang and P. Seeling, "Mininet: An Instant Virtual Network on Your Computer," in *Computing in Communication Networks* (Elsevier, 2020), 219–230.

52. S. Floyd, *Metrics for the Evaluation of Congestion Control Mechanisms* (IETF, 2008).

17518636, 2026, 1, Downloaded from https://ietresearch.onlinelibrary.wiley.com/doi/10.1049/cmu2.70141, Wiley Online Library on [09/04/2026]. See the Terms and Conditions (https://onlinelibrary.wiley.com/terms-and-conditions) on Wiley Online Library for rules of use; OA articles are governed by the applicable Creative Commons License